

Informe técnico - Primera entrega parcial del TPI (Food Store)

1. Qué se implementó en cada unidad

Arranqué del TP1 con el modelo: diagrama ER de 5 entidades donde detecté la N:M entre pedido y producto, que resolví con la intermedia pedido_producto. Lo pase a relacional con las FK y sus acciones justificadas (RESTRICT para no borrar clientes con facturas ni productos historicos, CASCADE para las líneas si se anula un ticket), y lo normalize hasta 3FN con las dependencias documentadas. El DDL quedó en Food Store.sql con lo propio del motor: ENUM forma_pago_enum, IDENTITY, TIMESTAMPTZ, UNIQUE en email, CHECKs de precios/stock/cantidad y los flags de baja lógica con DEFAULT.

En la Unidad 2 metí las reglas R1 (precio_unitario <= precio_lista) y R2 (fecha no futura) como funciones + triggers en PL/pgSQL, y reproducí 3 escenarios de concurrencia en dos sesiones (no repetible, fantasma y bloqueo) probando READ COMMITTED contra REPEATABLE READ. Todo con el protocolo de siempre: copia, transacción Manual y respaldo.

La Unidad 3 fue la saga de optimización: carga masiva de 200k en TP3, plan de indexado con EXPLAIN ANALYZE en TP3/TP4/TP5 (con mis documentadas no efectivas incluidas), vistas y la materializada de facturación en TP5, más el ranking con RANK() y los nunca vendidos con NOT EXISTS en TP4. Para cerrar el parcial me quedaban 3 huecos del checklist y los tapé en food-store-parcial/: la consulta con HAVING, el procedimiento con CALL y la demo de borrado lógico.

2. Cómo se probó el funcionamiento

Como en toda la cursada: nada se ejecuta sin spec de Kiro leído ni sin entender línea por línea lo que genero OpenCode, todo sobre food_store_copia y todo reversible. Acá me llevé la lección del parcial: el DDL también es transaccional en Manual. Creé la vista, la vi andar en una pestaña (50002) y en la otra me decía que no existía (42P01); había quedado sin commitear y un Revertir la borro. De ahí la regla que dejo escrita: el DDL estructural se commitea, solo los experimentos terminan en ROLLBACK. Las equivalencias, como siempre, con EXCEPT en ambos sentidos.

3. Qué resultados se obtuvieron (checklist de 9)

Obj	Evidencia	Resultado
1. ER	diagrama TP1	5 entidades, N:M detectada
2. Relacional	Food Store.sql	intermedia + FKs justificadas
3. 3FN/BCNF	desarrollo TP1	DF documentadas
4. DDL	schema + índices	ENUM/IDENTITY/TZ/UNIQUE
5. DML	queries + H1	JOIN, GROUP BY, HAVING, EXISTS, RANK
6. Vistas/func/proc	views, triggers, B0	3 vistas + MV + 2 func + 1 proc
7. Reglas	restricciones.sql	CHECK/UNIQUE y triggers R1-R2
8. Transacciones	informe_concurrencia	3 anomalías reproducidas
9. Borrado lógico	B1/B2/B3	CALL: 50001 y 0 filas; ROLLBACK: 50002

El HAVING me sorprendió: esperaba ~10 filas (carga pareja) y dio 6, o sea que filtra de verdad (30M/40M/50M pasan, el resto no). La demo de borrado quedó redonda: CALL sp_baja_producto(3), la vista pasa de 50002 a 50001 y el producto 3 da 0 filas; ROLLBACK y vuelve todo a 50002. El procedimiento además tira excepción si el producto no existe o ya esta dado de baja.

4. Consultas optimizadas y diferencias antes/después

Consulta	Antes	Después	X
TP3 Q3 precio	Seq Scan 3.394 ms	Bitmap 1.636 ms	2.07x
TP3 Q1	Bitmap 2.101 ms	Bitmap nuevo 1.430 ms	1.46x
TP4 Q-A / Q-B	Hash 430 / 152.9 ms	sigue en Hash	no efectivas
TP5 Q1	Seq Scan 43.185 ms	Index Scan 0.031 ms	1393x
TP5 Q2	Bitmap+Filter 9.581 ms	Index Cond 0.248 ms	38.6x
TP5 Q3	Seq Scan 19.779 ms	sigue Seq 14.065 ms	no efectiva
MV facturación	agregado ~430 ms	precargada 0.038 ms	~11300x

La lectura es la de siempre: el covering de TP4 no alcanzo porque para agregar todo hay que leer todo (Hash en memoria le gana a 200k lookups), y en TP5 el compuesto/parcial si alcanza porque el filtro recorta al 1-5%. Cuando recorta al 40% (Q3) el planificador vuelve a Seq Scan, y esta bien. El índice no es bueno o malo: es selectivo o no.

5. Uso de IA

Kiro para especificar y OpenCode para generar, como pide la catedra; no use otras IA. Acepté los índices compuestos/parciales, las vistas, la MV y el procedimiento. Descarté con justificación: el índice solo en forma_pago (cardinalidad 1), los covering de TP4 (no efectivos) y las explicaciones de planes que confundían costo con tiempo. Cada TP tiene su bitacora DUIA.

Anexo - Capturas de cierre de gaps

H1 - HAVING de facturación por categoría: 6 filas sobre 20M (Obj. 5).

```

SELECT c.nombre AS categoria, SUM(d.cantidad*d.precio_unitario) AS facturacion
FROM categoria c JOIN producto p ON p.id_categoria=c.id_categoria
JOIN pedido_producto d ON d.id_producto=p.id_producto
GROUP BY c.nombre HAVING SUM(d.cantidad*d.precio_unitario) > 20000000
ORDER BY facturacion DESC;

```

id	categoria	facturacion
1	Categoria masiva 4	50.000.000
2	Categoria masiva 9	50.000.000
3	Categoria masiva 3	40.000.000
4	Categoria masiva 8	40.000.000
5	Categoria masiva 2	30.000.000
6	Categoria masiva 7	30.000.000

B0 - CREATE PROCEDURE sp_baja_producto en 0.066 s (Obj. 6).

```
CREATE OR REPLACE PROCEDURE sp_baja_producto(p_id BIGINT) LANGUAGE plpgsql AS
$$ BEGIN UPDATE producto SET activo=FALSE WHERE id_producto=p_id AND activo=TRUE;
IF NOT FOUND THEN RAISE EXCEPTION 'producto % inexistente o ya dado de baja', p_id; END IF; END; $$;
```

Name	Value
Updated Rows	0
Execute time	0.066s
Start time	Wed Sep 23 10:44:39 GMT-03:00 2026
Finish time	Wed Sep 23 10:44:39 GMT-03:00 2026
Query	CREATE OR REPLACE PROCEDURE sp_baja_producto(p_id BIGINT) LANGUAGE plpgsql AS \$\$ BEGIN UPDATE producto SET activo=FALSE WHERE id_producto=p_id AND activo=TRUE; IF NOT FOUND THEN RAISE EXCEPTION 'producto % inexistente o ya dado de baja', p_id; END IF; END; \$\$

B1 - CALL sp_baja_producto(3): la vista pasa a 50001 (Obj. 9).

```
BEGIN;  
CALL sp_baja_producto(3);  
SELECT * FROM v_productos_vigentes WHERE id_producto=3;  
SELECT COUNT(*) FROM v_productos_vigentes;  
ROLLBACK;
```

Count	Count
1	50,001

B2 - El producto 3 ya no aparece en la vista: 0 filas (Obj. 9).

Script: C:\Users\Fabri\OneDrive\Documents\Default Project\sql\carga_masiva_tp3.sql

Connection: postgres

Type: PostgreSQL

URL: jdbc:postgresql://localhost:5432/postgres

Database: food_store_copia

Schema: public

id_producto	nombre_producto	precio_lista	stock	nombre_categoria
123	A2	1000	10	A2

GMT-03:00 es Editable Inserción inteligente 5:10:145

B3 - Tras el ROLLBACK: 50002 restaurado, base intacta (Obj. 8 y 9).

The screenshot shows the PostgreSQL command line interface. The command prompt is at the top, followed by the query editor where the command `SELECT COUNT(*) FROM v_productos_vigentes;` has been entered. The results window at the bottom shows a single row with the value `50002`.